Computability

Last modified: 2026-11-11 00:40:31 (PST)

Some problems have no algorithm at all, however much time is allowed. No Turing machine decides whether a program halts. The proof uses an argument called diagonalization, and the same argument shows that more time lets machines decide more problems. These results build on the models of computation page, which defines Turing machines, their descriptions as strings, and the universal machine.

Decidable problems

Definition 0.1 (Decidable problem)**.** A decision problem L is **decidable** if some Turing machine decides it: the machine halts on every input x, with output 1 if $x \in L$ and 0 if $x \notin L$. A problem that no Turing machine decides is *undecidable* (Arora and Barak 2009, sec. 1.5).

Example 0.1 (Decidable problems from P and NP)**.** The palindromes are decidable, because the machine of the palindrome example decides them. INDSET is decidable too, even though no polynomial-time algorithm for it is known: it is in NP, so a machine can run its verifier, with certificate length q as in the definition of NP, on each of the $2^{q(|x|)}$ possible certificates in turn, and accept if any certificate passes. That search takes exponential time, but it always halts. Decidability asks only whether a machine that always halts exists, not how long it takes.

An undecidable problem

Definition 0.2 (The diagonal problem)**.** The **diagonal problem** UC is the decision problem whose yes inputs are the strings w such that the machine M_w that w describes, run on its own description w, does not halt with output 1:

$$\mathrm{UC} \stackrel{\text{def}}{=} \{w \in \{0,1\}^* : M_w(w) \neq 1\},$$

where $M_w(w) \neq 1$ includes the case that M_w never halts on w (Arora and Barak 2009, sec. 1.5).

Example 0.2 (Machines in and out of the diagonal problem)**.** If w describes a machine that outputs 1 on every input, then $M_w(w) = 1$, so w is not in UC. If w describes a machine that outputs 0 on every input, or one that runs forever on every input, then w is in UC. If w describes the palindrome machine of the palindrome example, then w is in UC exactly when w is not itself a palindrome.

Theorem 0.1 (The diagonal problem is undecidable)**.** *The diagonal problem UC is undecidable (Arora and Barak 2009, sec. 1.5).*

Proof

Proof. Suppose a Turing machine M decides UC, and let w be a description of M, so that $M_w = M$. Then M halts on w with output 1 or 0.

- If $M(w) = 1$, then $w \in \mathrm{UC}$, because M decides UC. By Definition 0.2, $w \in \mathrm{UC}$ means $M_w(w) \neq 1$, that is, $M(w) \neq 1$, a contradiction.
- If $M(w) = 0$, then $w \notin \mathrm{UC}$, because M decides UC. By Definition 0.2, $w \notin \mathrm{UC}$ means $M_w(w) = 1$, that is, $M(w) = 1$, a contradiction.

Both cases are impossible, so no Turing machine decides UC. □

Remark 0.1 (The diagonal argument). Picture an infinite table with one row for each binary string w, one column for each binary string x, and the entry in row w and column x equal to $M_w(x)$, or to a mark for "runs forever". Each row lists everything one machine does. UC is built from the diagonal entries $M_w(w)$, and it disagrees with row w in column w for every w, so no row decides it. Arora and Barak (2009, chap. 3) use the same idea to prove the hierarchy theorems.

The halting problem

Definition 0.3 (The halting problem)**.** The **halting problem** HALT is the decision problem

$$\text{HALT} \stackrel{\text{def}}{=} \{\langle w, x\rangle : M_w \text{ halts on input } x\},$$

where $\langle w, x\rangle$ is the pair encoding of w and x, and M_w is the machine that w describes (Arora and Barak 2009, sec. 1.5).

Example 0.3 (Pairs in and out of the halting problem)**.** Suppose w describes a machine that halts at once if the first symbol of its input is 0, and otherwise moves its input head right forever. Then $\langle w, 0\rangle$ and $\langle w, 011\rangle$ are in HALT, and $\langle w, 1\rangle$ is not. Neither is $\langle w, x\rangle$ for the empty string x, because the first symbol the machine reads after $\triangleright$ is then a blank, not 0.

Theorem 0.2 (The halting problem is undecidable)**.** *The halting problem HALT is undecidable (Turing 1937; Arora and Barak 2009, sec. 1.5).*

Proof

Proof. Suppose a Turing machine H decides HALT. Build a machine D that, on input w, does the following:

1. Run H on $\langle w, w\rangle$.
2. If H outputs 0, then M_w does not halt on w, so $M_w(w) \neq 1$ and $w \in \text{UC}$: output 1.
3. If H outputs 1, then M_w halts on w. Run the universal machine U on $\langle w, w\rangle$, with w as the description and as the input, which halts with output $M_w(w)$. Output 0 if $M_w(w) = 1$, and 1 otherwise.

H halts on every input, and step 3 runs U only when M_w halts on w, so D halts on every input. Its output is 1 exactly when $M_w(w) \neq 1$, so D decides the diagonal problem UC. This contradicts Theorem 0.1, so no Turing machine decides HALT. □

Remark 0.2 (Undecidability in practice). Turing machines can simulate programs in ordinary programming languages (the Church-Turing thesis), so Theorem 0.2 says that no program can check, for every program and input, whether that program finishes. A job scheduler that runs users' model-fitting code cannot predict which jobs will loop forever, so it stops jobs with a time limit instead. Tools that always finish and check code for

infinite loops exist, but on some programs each one must answer "unknown", miss a loop, or flag a program that does halt. The proof that Kolmogorov complexity cannot be computed is a second example of an uncomputable function, proved by a different argument.

The time hierarchy theorem

Definition 0.4 (Time complexity class)**.** Let T be a function from the nonnegative integers to the positive real numbers. The class DTIME($T(n)$) is the set of decision problems L such that some Turing machine decides L and runs in time $T'(n)$ for some function T' with $T'(n) = O(T(n))$ (big-O notation). Arora and Barak (2009, sec. 1.6) require a running time of at most $c\,T(n)$ for a constant c and every n. Because $T(n) > 0$, the two definitions agree: a larger c covers the finitely many n below the threshold of $O(\cdot)$.

Example 0.4 (Palindromes and the class P)**.** The palindrome machine runs in time $3n+4$, which is at most $7n$ for every $n \geq 1$ and so is $O(n)$, so PAL is in DTIME(n) (time complexity class). The class P is the union of DTIME(n^c) over all constants $c \geq 1$.

Theorem 0.3 (More time decides more problems)**.** *The time complexity class DTIME(n) is a proper subset of DTIME($n^{1.5}$) (Arora and Barak 2009, sec. 3.1).*

Proof

Proof. Every problem in DTIME(n) is in DTIME($n^{1.5}$), because $n = O(n^{1.5})$. For the other half, let U be the universal machine, and build a machine D that, on an input x of length n, does the following:

1. Copy the pair encoding $\langle x, x\rangle$ onto a work tape.
2. Compute the number $t = \lfloor n^{1.4} \rfloor$ in binary on another work tape.
3. Run U on $\langle x, x\rangle$, which runs the machine that x describes on input x, and subtract 1 from t after each step of U. Stop U when t reaches 0.
4. If U halted within those steps with output $b \in \{0, 1\}$, output $1 - b$. Otherwise output 0.

Steps 1 and 2 take $O(n \log_2 n)$ steps: D writes $\langle x, x\rangle$, counts the symbols of x in binary, then does arithmetic on numbers of $O(\log_2 n)$ digits. Step 3 runs U for at most $n^{1.4}$ steps, and each subtraction takes $O(\log_2 n)$ steps, so step 3 takes $O(n^{1.4} \log_2 n)$ steps,

which is $O(n^{1.5})$ because $\log_2 n = O(n^{0.1})$. So D halts on every input, and the problem L that D decides is in DTIME$(n^{1.5})$.
Suppose L is also in DTIME(n). Then some Turing machine M decides L, and there are a constant $c \geq 1$ and an integer n_0 such that M halts within $c\,n$ steps on every input of length $n \geq n_0$. Let x be an input of length $n \geq n_0$, and let w be any description of M. By **?@thm-universal-tm**, U halts with output $M(x)$ within $C\,c\,n\log_2(c\,n)$ steps when it runs M_w on x, where C depends on M but not on w or x. That bound is $O(n\log_2 n)$, so there is an $n_1 \geq n_0$ such that the bound is less than $\lfloor n^{1.4} \rfloor$ for every $n \geq n_1$. M has infinitely many descriptions, so it has a description x of length at least n_1. On that input, U finishes within the $\lfloor n^{1.4} \rfloor$ steps that D allows, with output $b = M(x)$, so $D(x) = 1 - M(x) \neq M(x)$. But M and D both decide L, so $M(x) = D(x)$, a contradiction. So L is not in DTIME(n). □

Theorem 0.4 (The time hierarchy theorem)**.** *Let f and g be time-constructible functions with $f(n)\log_2 f(n) = o(g(n))$ (little-o notation). Then the time complexity class DTIME$(f(n))$ is a proper subset of DTIME$(g(n))$ (Arora and Barak 2009, sec. 3.1).*

Remark 0.3 (Proof and consequences of the time hierarchy theorem). A refinement of the proof of Theorem 0.3 proves Theorem 0.4. It uses the same diagonal machine, but with a clock that costs a constant number of steps per simulated step on average, in place of the $O(\log_2 n)$ steps per subtraction in the proof of Theorem 0.3. That proof could afford the extra $\log_2 n$ factor only because $n^{1.4}$ and $n^{1.5}$ differ by a power of n. The $\log_2 f(n)$ factor in the hypothesis is the overhead of the universal machine in the universal machine theorem. Since $n^c \log_2 n^c = o(n^{c+1})$, DTIME$(n^c)$ is a proper subset of DTIME(n^{c+1}) for every integer $c \geq 1$, so the class P is not equal to DTIME(n^c) for any single constant c.

References

Arora, Sanjeev, and Boaz Barak. 2009. *Computational Complexity: A Modern Approach*. Cambridge University Press. https://doi.org/10.1017/CBO9780511804090.

Turing, A. M. 1937. "On Computable Numbers, with an Application to the Entscheidungsproblem." *Proceedings of the London Mathematical Society* s2-42 (1): 230–65. https://doi.org/10.1112/plms/s2-42.1.230.